BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

C PROGRAMMING

Unit 3

I/O Operations in C

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3
	1.1 Objectives	-	-	
2	Formatted input and output	1, 2	1	4 - 10
3	Interactive Programming	-	-	11 - 12
4	Summary	-	-	13
5	Terminal Questions	-	-	14
6	Answers to Self Assessment Questions	-	-	15
7	Answers to Terminal Questions	-	-	15
8	Exercises	-	-	15

1. INTRODUCTION

The printf and scanf functions are commonly used in the C programming language for input and output operations. printf formats and prints output in C, using format specifiers like %d and %s. scanf reads formatted input, requiring variable addresses as arguments, e.g., scanf("%d", &num);. Maintaining specifier alignment between printf and scanf is vital. Careful input handling is essential to thwart security risks, such as buffer overflows. These functions play a fundamental role in robust input/output management in C programming.

The functions scanf() and printf() permit the transfer of single character, numerical values and strings; the functions gets() and puts() facilitate the input and output of strings. These functions can be accessed within a program by including the header file stdio.h.

1.1. Objectives:

At studying this unit, you should be able to:

- *Understand and implement the printf function in C for formatted output*
- *Gain proficiency in using scanf for formatted input in C programs*

2. FORMATTED INPUT AND OUTPUT

Input data can be entered into the computer from a standard input device by means of the standard C library function **scanf().** This function can be used to enter any combination of numerical values, single character and strings. The function returns the number of data items that have been entered successfully.

The syntax of scanf() function is as follows:

scanf(control string, arg1, arg2, ...argn)

where control string refers to a string containing certain required formatting information, and *arg1, arg2,…, argn* are arguments that represent the individual input data items. The arguments represent pointers that indicate addresses of the data items within the computer's memory.

The control string consists of control characters, whitespace characters, and non-whitespace characters. The control characters are preceded by a % sign, and are listed in Table 3.1.1.

Table 3.1.1: Control characters and its explanations

Control Character	Explanation
%c	a single character
%d	a decimal integer
%i	an integer
%e, %f, %g	a floating-point number
%o	an octal number
%s	a string
%x	a hexadecimal number

%p	a pointer
%n	an integer equal to the number of characters read so far
%u	an unsigned integer
%[]	a set of characters
%%	a percent sign

scanf() reads the input, matching the characters from format. When a control character is read, it puts the value in the next variable. Whitespaces (tabs, spaces, etc) are skipped. Non-whitespace characters are matched to the input, then discarded. If a number comes between the % sign and the control character, then only that many characters will be entered into the variable. If **scanf()** encounters a set of characters, denoted by the %[] control character, then any characters found within the brackets are read into the variable. The return value of **scanf()** is the number of variables that were successfully assigned values, or EOF if there is an error.

Program : Program to use scanf() to read integers, floats, characters and strings from the user.

```
#include<stdio.h>
main()
{
int i;
float f;
char c;
char str[10];
scanf("%d %f %c %s", &i, &f, &c, str);
printf("%d %f %c %s", i, f, c, str);
}
```

Execute this program and observe the result.

Note that for a **scanf()** function, the addresses of the variable are used as the arguments for an **int, float and a char** type variable. But this is not true for a string variable because a string name itself refers to the address of a string variable.

A *control* character is used to enter strings to string variables. A string that includes whitespace characters can not be entered. There are ways to work with strings that include whitespace characters. One way is to use the **getchar()** function within a loop. Another way is to use **gets()** function which will be discussed later.

It is also possible to use the **scanf()** function to enter such strings. To do so, the s-control character must be replaced by a sequence of characters enclosed in square brackets, designated as [...]. Whitespace characters may be included within the brackets, thus accommodating strings that contain such characters.

Example:

```
#include<stdio.h>
main()
{
char str[80];
...
scanf("%[ ABCDEFGHIJKLMNOPQRST]", str);
...
}
```

This example illustrates the use of the scanf() function to enter a string consisting of uppercase letters and blank spaces. Please note that if you want to allow lowercase letters to be entered, all the lowercase letters (i.e. from a-z) must be included in the list of control string.

Formatted Output

Output data can be written from the computer onto a standard output device using the library function **printf()**. This function can be used to output any combination of numerical

values, single characters and strings. It is similar to the input function **scanf(),** except that its purpose is to display data rather than enter into the computer.

The syntax of the **printf** function can be written as follows:

printf(control string, arg1, arg2, ..., argn)

where control string refers to a string that contains formatting information, and arg1, arg2, …, argn are arguments that represent the individual output data items. The arguments can be written as constants, single variable or array names, or more complex expressions.

Examples:

```
printf("Hello, world!\n");

printf("i is %d\n", i);

printf("%d", 10);

printf("%d", i+j);
```

The first statement simply displays the string given as argument to the printf() function. In the second statement, printf() function replaces the two characters %d with the value of the variable i. In the third statement the argument to be printed is a constant and in the fourth, the argument is an expression.

There are quite a number of format specifiers for printf(). Some of them are listed in Table 3.1.2.

Table 3.1.2: Format specifiers for printf().

%d	Print an int argument in decimal
%ld	print a long int argument in decimal
%c	print a character
%s	print a string
%f	print a float or double argument
%e	same as %f, but use exponential notation
%g	use %e or %f, whichever is better
%o	print an int argument in octal (base 8)
%x	print an int argument in hexadecimal (base 16)
%%	print a single %

It is also possible to specify the width and precision of numbers and strings as they are inserted ; For example, a notation like %3d means to print an int in a field at least 3 spaces wide; a notation like %5.2f means to print a float or **double** in a field at least 5 spaces wide, with two places to the right of the decimal.)

To illustrate with a few more examples: the call

printf("%c %d %f %e %s %d%%\n", '3', 4, 3.24, 66000000, "nine", 8); would print

3 4 3.240000 6.600000e+07 nine 8%

The call

printf("%d %o %x\n", 100, 100, 100);

would print

```
100 144 64
```

Successive calls to printf() just build up the output a piece at a time, so the calls

```
printf("Hello, ");
printf("world!\n");
```

would also print Hello, world! (on one line of output).

Earlier we learned that C represents characters internally as small integers corresponding to the characters' values in the machine's character set (typically ASCII). This means that there isn't really much difference between a character and an integer in C; most of the difference is in whether we choose to interpret an integer as an integer or a character. printf is one place where we get to make that choice: %d prints an integer value as a

string of digits representing its decimal value, while %c prints the character corresponding to a character set value. So the lines

```
char c = 'A';
int i = 97;
printf("c = %c, i = %d\n", c, i);
```

would print c as the character A and i as the number 97. But if, on the other hand, we called

```
printf("c = %d, i = %c\n", c, i);
```

we'd see the decimal value (printed by %d) of the character 'A', followed by the character (whatever it is) which happens to have the decimal value 97.

You have to be careful when calling **printf().** It has no way of knowing how many arguments you've passed it or what their types are other than by looking for the format specifiers in the format string. If there are more format specifiers (that is, more % signs) than the arguments, or if the arguments have the wrong types for the format specifiers, **printf()** can misbehave

badly, often printing nonsense numbers or (even worse) numbers which mislead you into thinking that some other part of your program is broken.

Because of some automatic conversion rules which we haven't covered yet, you have a small amount of latitude in the types of the expressions you pass as arguments to **printf().** The argument for %c may be of type **char** or **int**, and the argument for %d may be of type **char** or **int**. The string argument for %s may be a string constant, an array of characters, or a pointer to some characters. Finally, the arguments corresponding to %e, %f, and %g may be of **types float** or **double**. But other combinations do not work reliably: %d will not print a long int or a **float** or a **double**; %ld will not print an int; %e, %f, and %g will not print an **int**.

SELF-ASSESSMENT QUESTIONS - 1

1. The _____ string consists of control characters, whitespace characters, and non-whitespace characters.
2. The control string used to read a hexadecimal character is_____.
3. scanf() functions needs address of the data item to be read as the argument. (True/False).
4. The output of the following statement is _____. printf("%d %o %x\n", 64, 10, 75);
5. To print an int argument in octal, you can use _____ format string.
6. The output of the following program segment is _____.

```
int a=97;

printf("%c", a);
```

2. INTERACTIVE PROGRAMMING

Creating interactive dialog between the computer and the user is a modern style of programming. These dialogs usually involve some form of question-answer interaction, where the computer asks the questions and the user provides the answer, or vice versa. In C, such dialogs can be created by alternate use of the scanf() and printf() functions.

Program : Program to calculate the simple interest

```
#include<stdio.h>
main()
{
/* Sample interactive program*/
float principle, rate, time, interest;
printf(" Please enter the principle amount: ");
scanf("%f", &principle);
printf(" Please enter the rate of interest: ");
scanf("%f", &rate);
printf(" Please enter the period of deposit: ");
scanf("%f", &time);
interest=principle*rate*time/100.0;
printf("Principle=%7.2f\n", principle);
printf("Rate of interest=%5.2f\n",rate);
printf("Period of deposit=%5.2f\n", time);
printf("Interest=%7.2f\n", interest);
}
```

Execute the above program and observe the result.

Example Input:

Please enter the principle amount: 1000

Please enter the rate of interest: 5

Please enter the period of deposit: 2

Corresponding Output:

Principle=1000.00

Rate of interest= 5.00

Period of deposit= 2.00

Interest= 100.00

INSPIRED BY LIFE

3. SUMMARY

scanf() and **printf()** are the two formatted input/output functions. These functions can handle characters, numerical values and strings as well. In C, printf formats output using placeholders like %d or %s, while scanf handles formatted input with specifiers such as %d for variables, ensuring data type alignment. For example, scanf("%d", &num); reads an integer. Secure input practices, like validating and sanitizing user inputs, are crucial to prevent vulnerabilities like buffer overflows. Proper specifier matching between printf and scanf is essential for program reliability. Additional concepts include escape sequences for special characters and modifiers for field width and precision. Understanding these functions is fundamental for controlled, secure, and versatile input/output operations in C, contributing to the development of robust and effective programs.

INSPIRED BY LIFE

4. TERMINAL QUESTIONS

1. Explain the role of format specifiers in the printf function in C. Provide examples of different format specifiers and their corresponding data types.
2. Describe how the scanf function is used for input in C. Include examples with different format specifiers and discuss the importance of using the correct specifiers for variable types.
3. Discuss the significance of the & (ampersand) operator in the context of the scanf function. Provide an example to illustrate its usage.

INSPIRED BY LIFE

5. ANSWERS TO SELF ASSESSMENT QUESTIONS

1. Control
2. %x
3. True
4. 64, 12, 4B
5. %o
6. a

6. ANSWERS TO TERMINAL QUESTIONS

1. Refer section 2
2. Refer section 2
3. Refer section 2

7. EXERCISES

1. Write C that calculates the factorial of a given number.
2. Write a C program that takes two integers as input from the user, calculates their sum and product, and prints the results.